

The Complete Lifecycle of a FastAPI Application

From `fastapi dev main.py` to a response in your browser — every step, explained.

IN THIS ARTICLE

1. Phase 1 — You run `fastapi dev main.py`
2. Phase 2 — Python imports and app creation
3. Phase 3 — Route registration (how `@app.get` works)
4. Phase 4 — Uvicorn starts the ASGI server
5. Phase 5 — Startup events & lifespan
6. Phase 6 — A request arrives
7. Phase 7 — Middleware chain
8. Phase 8 — Routing, dependencies & your function
9. Phase 9 — Response construction & serialization
10. Phase 10 — Shutdown

1 You run `fastapi dev main.py`

When you type `fastapi dev main.py` in your terminal and press Enter, you are running the **FastAPI CLI** — a command-line tool that ships with FastAPI since version 0.111. The CLI is not the web server itself; it is a thin launcher.

Here is what it does immediately:

01 Parse the argument

It reads `main.py` as the target file, and `dev` as the mode (development). In dev mode, auto-reload is enabled by default.

02 Locate the FastAPI app object

The CLI scans `main.py` looking for a variable that is an instance of `FastAPI`. By convention it looks for a variable named `app`. This is what becomes the *ASGI application*.

03 Invoke Uvicorn

It (FastAPI CLI) calls Uvicorn under the hood, roughly equivalent to: `uvicorn main:app --reload --host 127.0.0.1 --port 8000`. Uvicorn is the actual ASGI server that will handle connections.

ASGI? ASGI stands for *Asynchronous Server Gateway Interface*. It is a standard protocol (a Python spec) that defines how a web server communicates with a Python web application. FastAPI is an ASGI framework. Uvicorn is an ASGI server. They speak the same protocol.

2 Python imports and app creation

Uvicorn now needs to import your module. It runs Python's import machinery on `main.py`. This causes **every line at the module level** to execute, top to bottom.

MAIN.PY

```
from fastapi import FastAPI
```

```
app = FastAPI()           # ← this line runs immediately on import
```

```
@app.get("/hello")       # ← this also runs immediately on import
async def say_hello():
    return {"message": "Hello!"}
```

The line `app = FastAPI()` creates a **FastAPI instance**. Internally, this object holds:

- A router (`APIRouter`)
An empty registry that will store all your route definitions. Think of it as a big dictionary: *path + method* → *handler function*.
- A middleware stack
An ordered list of middleware layers that every request and response will pass through.
- An OpenAPI schema generator
FastAPI automatically reads your route definitions and type hints to generate `/docs` (Swagger UI) and `/openapi.json` for free.

→ Exception handlers

Default handlers for 404 Not Found, 422 Validation Error, and 500 Internal Server Error are registered at startup.

3 Route registration — how `@app.get` works

This is the part most beginners find magical. Let's demystify it completely.

When Python processes your file top-to-bottom, it hits this line:

```
@app.get("/hello")
async def say_hello():
    return {"message": "Hello!"}
```

A **decorator** in Python is just syntactic sugar. The code above is exactly equivalent to:

```
async def say_hello():
    return {"message": "Hello!"}

say_hello = app.get("/hello")(say_hello)
# app.get("/hello") returns a decorator function
# that decorator receives say_hello as its argument
```

The key insight: `@app.get("/hello")` is called immediately when the file is imported, not when a request arrives. Your function `say_hello` is stored inside FastAPI's router at import time. No HTTP request has happened yet.

Internally, `app.get()` calls `self.router.add_api_route()`, which does the following:

01 Wrap your function in a Route object

FastAPI creates an `APIRoute` object that stores: the path (`/hello`), the HTTP method (`GET`), your function reference, and all the metadata from your type hints.

02 Inspect type hints with `inspect` + Pydantic

FastAPI uses Python's `inspect` module to read every parameter of your function. It builds a Pydantic model from those hints to validate inputs later. This analysis happens *right now at import time*, not per-request.

03 Compile a path regex

Path parameters like `/items/{item_id}` are turned into compiled regular expressions so matching is fast at request time.

04 Register in the router list

The completed route is appended to `app.routes`. FastAPI checks routes in order, so the first matching route wins.

Analogy: Think of a restaurant. When the restaurant first opens (*import time*), the chef prepares the menu — writes down every dish, its ingredients, how long it takes to cook. The menu (route table) is fully ready before any customer (*HTTP request*) walks in. When a customer orders "pizza", the waiter doesn't figure out what pizza is — they just look it up in the pre-built menu and hand it to the right chef.

4 Uvicorn starts the ASGI server

After your module is imported and all routes are registered, Uvicorn takes over. It starts an event loop using `asyncio` and binds a TCP socket to the configured host and port (default: `127.0.0.1:8000`).

01 Create the asyncio event loop

Uvicorn starts Python's asyncio event loop. All async code in your application — including your `async def` route handlers — will run inside this loop. It is single-threaded but concurrently handles thousands of connections via cooperative multitasking.

02 Bind a socket and start listening

Uvicorn opens a TCP socket on port 8000. The OS kernel now accepts incoming TCP connections at the network level. Your terminal shows: `Uvicorn running on http://127.0.0.1:8000`.

03 Call your app's `lifespan`

Before serving any request, Uvicorn calls the ASGI lifespan protocol on your app, triggering startup events. (See Phase 5.)

dev vs production: `fastapi dev` uses `--reload`, which makes Uvicorn watch your files for changes and restart automatically. In production you'd use `fastapi run` (no reload) or run Uvicorn behind a process manager like Gunicorn with multiple worker processes.

5 Startup events & lifespan

Before your app handles any request, FastAPI triggers the **startup phase**. This is where you initialize shared resources: database connection pools, ML models, caches — anything that should be created once and reused across requests.

The modern way to do this is with a **lifespan context manager**:

LIFESPAN PATTERN (RECOMMENDED)

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # — STARTUP —————
    print("Starting up: connecting to DB...")
    app.state.db = await connect_database()

    yield  # ← app is now live, serving requests

    # — SHUTDOWN —————
    print("Shutting down: closing DB...")
    await app.state.db.close()

app = FastAPI(lifespan=lifespan)
```

Everything before the `yield` runs at startup. Everything after it runs at shutdown. The app only begins accepting HTTP requests after the startup section completes successfully.

app.state is a simple namespace on the FastAPI object where you can store arbitrary data (DB connections, config, etc.) that persists for the lifetime of the app and can be accessed from any request handler via `request.app.state`.

6 A request arrives

A user visits `http://localhost:8000/hello` in their browser. Here is exactly what happens at the network level:

01 TCP connection & HTTP parsing (Uvicorn)

Uvicorn accepts the TCP connection. It reads the raw bytes from the socket and parses the HTTP/1.1 (or HTTP/2) request: method, path, headers, body.

02 Build the ASGI scope dict

Uvicorn constructs a Python dictionary called the **scope**. It looks like: `{"type": "http", "method": "GET", "path": "/hello", "headers": [ ... ], ... }`. This dict is the standard ASGI way to describe a request.

03 Call the ASGI app

Uvicorn calls `await app(scope, receive, send)`. This is the *one and only* entry point into your FastAPI application. `receive` is a coroutine to read the request body; `send` is a coroutine to write response bytes back.

Who calls your route function? Not Uvicorn directly. The call chain is: **Uvicorn → FastAPI app → Middleware stack → Router → Route handler → Your function**. Uvicorn only ever knows about the top-level ASGI interface. Your function is several layers deep.

7 The middleware chain

Before your route function is even considered, the request passes through every **middleware** layer, from outermost to innermost. Think of them as a stack of onion layers that the request must peel through going in, and peel back through going out.

MIDDLEWARE EXAMPLE

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
```

```
    allow_origins=["*"],  
)
```

Each middleware is an ASGI callable itself. When one middleware wants to proceed, it calls `await call_next(request)`, which hands the request to the next layer in the chain. The last layer in the chain is FastAPI's own router.

→ Request direction (in)

Middleware 1 → Middleware 2 → Middleware N → FastAPI Router. Each middleware can inspect or modify the request before passing it on, or short-circuit and return a response immediately (e.g., auth middleware rejecting an unauthenticated request).

← Response direction (out)

Once the route handler produces a response, it travels back out through the same middleware stack in reverse order. Middleware can modify the response too — adding CORS headers, compressing the body, logging the status code.

Middleware registered last with `add_middleware()` actually runs *first* in the request direction, because FastAPI wraps them in reverse registration order. This is a common source of confusion — keep it in mind when ordering middleware matters (e.g., logging should be outermost to capture everything).

8 Routing, dependencies & your function gets called

After clearing the middleware stack, the request reaches FastAPI's **router**. This is where your `@app.get("/hello")` registration from Phase 3 finally pays off.

01 Path matching

The router walks its list of routes and matches the incoming path (`/hello`) against the pre-compiled regex for each route. The first match wins. If no route matches, a 404 response is returned immediately.

02 Build a `Request` object

FastAPI wraps the raw ASGI scope into a `Request` object — a convenient Python interface with properties like `request.headers`, `request.query_params`, `await request.json()`, etc.

03 Resolve dependencies (`Depends`)

FastAPI inspects your function signature. Any parameter typed with `Depends(some_function)` is treated as a dependency. FastAPI calls the dependency function first (recursively resolving its own dependencies), and injects the result as an argument. Dependencies can also `yield` for cleanup after the response.

04 Extract & validate parameters

Using the Pydantic models built at import time (Phase 3), FastAPI extracts path params, query params, headers, cookies, and the request body. It validates types, runs validators, and coerces values. If validation fails, a `422 Unprocessable Entity` response is returned automatically — you don't write any of this validation code.

05 Call YOUR function

Finally! FastAPI calls `await say_hello(**validated_params)`. If your function is `async def`, it is awaited directly on the event loop. If it is a plain `def`, FastAPI runs it in a thread pool executor to avoid blocking the event loop.

FULL EXAMPLE SHOWING WHAT FASTAPI RESOLVES AUTOMATICALLY

```
from fastapi import FastAPI, Depends, Query
from pydantic import BaseModel

async def get_db():          # dependency
    db = await connect()
    yield db
    await db.close()        # cleanup runs after response

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/{item_id}")
async def create_item(
    item_id: int,             # ← path param, auto-extracted + int-coerced
    q: str = Query(None),    # ← query param (?q=...)
    body: Item = ...,        # ← request body, validated via Pydantic
    db = Depends(get_db),    # ← dependency injected by FastAPI
):
    return {"id": item_id, "name": body.name}
```

Who calls your function? FastAPI's internal `run_endpoint_function()` (inside `routing.py`) is the function that actually calls yours. It is invoked by the matched `APIRoute`'s `handle()` method, which is triggered by the router when a path match is found.

9 Response construction & serialization

Your function returned a value. Now FastAPI must turn that Python object into bytes over the wire.

01 Serialize the return value

If you returned a Pydantic model, a dict, a list, or a plain Python type, FastAPI serializes it to JSON using Pydantic's `.model_dump()` + `jsonable_encoder()`. This also filters fields if you declared a `response_model=`.

02 Wrap in a `Response` object

FastAPI wraps the JSON bytes in a `JSONResponse` (the default) with status 200, and sets the `Content-Type: application/json` header. You can return a custom `Response` directly to override all this.

03 Response travels back through middleware

The Response object bubbles back out through the middleware stack (Phase 7, in reverse). Each middleware can modify it — adding headers, compressing content, etc.

04 Uvicorn sends bytes to the client

FastAPI calls the ASGI `send` coroutine to emit the HTTP response headers, then the body bytes. Uvicorn writes these to the TCP socket. The browser receives them and renders the JSON.

05 Dependency cleanup (`yield` teardown)

After the response is sent, FastAPI resumes any dependencies that used `yield`. This is where database connections are closed, temp files cleaned up, etc. The client already has their response — this runs in the background.

10 Shutdown

When you press `Ctrl+C`, Uvicorn catches the `SIGINT` signal. It does not kill the process immediately — it initiates a graceful shutdown:

01 Stop accepting new connections

The listening socket is closed so no new requests can come in.

02 Finish in-flight requests

Uvicorn waits for all currently-running request handlers to complete (up to a configurable timeout).

03 Trigger the lifespan shutdown

FastAPI's ASGI lifespan sends a `lifespan.shutdown` event, which resumes your `lifespan` context manager after the `yield`. Your cleanup code runs — closing DB pools, flushing caches, etc.

04 Event loop closes, process exits

The asyncio event loop is stopped and closed. The Python process exits cleanly.

✓ The full picture — at a glance

CLI STARTS <code>fastapi dev</code>	IMPORTS MODULE <code>Python import</code>	ROUTES BUILT <code>@app.get() time</code>	SERVER BINDS <code>:8000 socket</code>
STARTUP RUNS <code>lifespan yield</code>	REQUEST IN <code>ASGI scope</code>	MIDDLEWARE <code>onion layers</code>	ROUTE MATCH <code>regex → route</code>
VALIDATE <code>Pydantic v2</code>	YOUR FUNCTION! <code>await handler()</code>	SERIALIZE <code>JSONResponse</code>	BYTES SENT <code>ASGI send()</code>